Exercise: CNN Image Classifier With Preprocessing

Dataset: Chest X-Ray Images (Pneumonia) — Kaggle, ~1.2GB, binary classification (NORMAL vs PNEUMONIA)

Why this dataset: it looks simple (2 classes) but has three real preprocessing problems baked in, which is the point of this exercise:

1. Images are inconsistent sizes and some are grayscale/some RGB
2. The classes are **imbalanced** (roughly 3x more PNEUMONIA images than NORMAL)
3. The provided `val/` folder only has 16 images — way too small to trust, so you'll need to build your own validation split

This mirrors what you'll actually run into in real datasets — not the clean, balanced, pre-sized toy sets most tutorials use.

Setup

```
pip install kaggle tensorflow pillow scikit-learn matplotlib
```

Get your Kaggle API token (Account → Create New API Token on kaggle.com), then:

```
mkdir -p ~/.kaggle && mv kaggle.json ~/.kaggle/
kaggle datasets download -d paultimothymooney/chest-xray-pneumonia
unzip chest-xray-pneumonia.zip -d chest_xray_data
```

You should end up with `chest_xray_data/chest_xray/{train,test,val}/{NORMAL,PNEUMONIA}/`.

Learning objectives

By the end you should be able to:

- Build an image preprocessing pipeline that handles inconsistent inputs (size, color mode)
- Diagnose and correct for class imbalance
- Build and train a CNN from scratch (not a pretrained model — that's a good follow-up exercise later)
- Evaluate a classifier properly when imbalance is present (accuracy alone will lie to you here)

Tasks

Task 1 — Inspect before you touch anything

- Count images per class in `train/` and `test/`.
- Sample and display 5 images per class. Check: are they all the same size? Same color mode (RGB vs grayscale)?
- Write down the class imbalance ratio — you'll need it in Task 3.

Checkpoint: you should find PNEUMONIA has roughly 3x as many training images as NORMAL, and that image dimensions vary (they are *not* pre-resized).

Task 2 — Build a proper train/validation split

- Ignore the provided `val/` folder (too small — 16 images won't give reliable signal).
- Split the `train/` folder yourself: 85% train / 15% validation, **stratified by class** so the ratio is preserved in both splits.
- Keep `test/` untouched — that's your final, only-look-at-it-once evaluation set.

Task 3 — Preprocessing pipeline

Build a pipeline that:

- Resizes every image to a fixed size (150×150 is a reasonable start)
- Converts all images to a single consistent color mode (grayscale is fine and faster here, since X-rays carry no meaningful color information)
- Normalizes pixel values to [0, 1]
- Applies data augmentation **to the training set only** (never augment validation/test): random rotation (~10–15°), zoom, horizontal flip, slight brightness variation
- Computes class weights from the imbalance ratio you found in Task 1, to pass into training later

Why this order matters: if you compute class weights *after* augmenting, or augment your validation set, your numbers will be misleading. Think about why before moving on.

Task 4 — Build the CNN

Design a CNN with:

- At least 3 convolutional blocks (Conv2D → BatchNorm → ReLU → MaxPool)
- Dropout somewhere before the final dense layers (this dataset is small enough to overfit fast)
- A final sigmoid output (binary classification)

Don't reach for a pretrained model yet — building one from scratch first will make it much clearer later *why* transfer learning helps.

Task 5 — Train with the right settings

- Use the class weights from Task 3 in your loss function/fit call.
- Add early stopping on validation loss (patience of ~3–5 epochs).
- Train for up to 20–30 epochs and let early stopping cut it off.

Task 6 — Evaluate properly

- Report accuracy **and** precision, recall, F1 per class, and a confusion matrix on the held-out `test/` set.
- Specifically answer: what's the recall on the NORMAL class? (This is the class most likely to suffer from the imbalance — a model can get deceptively high accuracy by just leaning toward predicting PNEUMONIA.)
- In a medical context, is a false negative (missing real pneumonia) or a false positive worse? Which one is your model making more of?

Bonus (optional, harder)

- Try training the same architecture *without* class weights and compare the confusion matrix — quantify how much the imbalance handling actually helped.
- Try swapping in a pretrained base (e.g., MobileNetV2) with frozen weights + your own classification head, and compare performance and training time against your from-scratch CNN.

What "done" looks like

A short writeup (few sentences is enough) covering:

- Your final test-set precision/recall/F1 per class
- Whether class weighting changed your confusion matrix meaningfully
- One thing that surprised you about the preprocessing step